

GPU-Accelerated Anisotropic Stencil Filter for Real-Time Edge Detection

Luke Bagan¹, Jack Vaught¹, Yi Wang¹

¹Department of Mechanical Engineering, University of South Carolina

Abstract — We present the Anisotropic Stencil Filter (ASF), a GPU-native edge detection operator that computes image gradients through orientation-selective polynomial fitting over large pixel neighborhoods. The filter models local image intensity with a high-order two-dimensional Taylor expansion, sweeps a set of candidate edge orientations, and selects the one producing the strongest normal derivative response. Unlike conventional gradient operators that rely on small fixed kernels, the ASF incorporates hundreds of pixels into each gradient estimate, providing inherent noise suppression without pre-smoothing. A key design contribution is the *fused stencil* formulation: the multi-point polynomial fitting operation is algebraically collapsed into a single precomputed weight vector per orientation, eliminating redundant memory accesses and enabling the entire filter to be applied as one gather-multiply-sum operation per pixel. This reformulation achieves up to 24× speedup and 311× reduction in GPU memory consumption compared to a naive implementation, while preserving identical output. On standard benchmarks (BSDS500, BIPED, UDED), the ASF matches or exceeds the accuracy of all classical non-learned edge detectors and approaches the performance of supervised deep learning models — without any training data. The implementation uses a custom Triton kernel optimized for modern GPU architectures, processing 1280×720 images in under 50 ms on an NVIDIA A100.

1 Introduction

Edge detection remains one of the most fundamental operations in computer vision. It underpins object detection pipelines, drives contour-based image segmentation, enables autonomous navigation through structured environments, and provides the geometric primitives on which industrial inspection and dimensional measurement systems depend [1], [2]. Despite more than five decades of sustained research, the core challenge persists. Real-world images present intensity transitions that are corrupted by sensor noise, obscured by fine texture, and modulated by spatially varying illumination, and no single operator has proven universally optimal across all of these conditions.

The earliest and still most widely deployed gradient operators are the small fixed-kernel convolutions introduced in the 1960s and 1970s. The Roberts Cross operator estimates gradients along two diagonal directions using a 2×2 stencil. The Prewitt filter [3] and the Sobel filter [4] extend the support region to 3×3 , incorporating a modest smoothing component orthogonal to the differentiation axis. These operators are trivially fast, require no parameters, and are available in every major image processing library. However, their small spatial support is also their principal limitation. With only nine pixels contributing to each gradient estimate, any noise within the window directly corrupts the output. Extending the kernel to 5×5 or 7×7 provides marginal improvement, but the hand-designed weight structures remain fixed and cannot adapt to the local edge orientation or the prevailing noise level in a given image region.

The Canny detector [1] addressed the noise sensitivity of small kernels by introducing a multi-stage pipeline. A Gaussian pre-smoothing step suppresses high-frequency noise before gradient computation, and subsequent non-maximum suppression and hysteresis thresholding stages refine the detected contours into thin, connected edge maps. The Canny detector became the dominant classical method and remains a standard baseline in the literature. Nevertheless, the Gaussian smoothing stage introduces a fundamental trade-off. Stronger smoothing suppresses more noise but simultaneously blurs weak edges and low-contrast boundaries, displacing them from their true locations. Furthermore, the gradient orientation at each pixel is computed via an arctangent of two cardinal finite differences, which propagates quantization and noise errors from the x and y channels into every intermediate angle. These limitations are well documented but difficult to overcome within the Canny framework, because the smoothing and differentiation stages are decoupled by design.

The past decade has seen a decisive shift toward deep learning approaches for edge detection. Holistically-Nested Edge Detection (HED) [5] demonstrated that a fully convolutional network with multi-scale side outputs could learn rich boundary representations from human-annotated ground truth. Subsequent architectures have pushed benchmark scores steadily higher. DexiNed [6] introduced dense extreme inception blocks for training

without ImageNet pre-training. PIDINet [7] integrated classical pixel-difference convolutions into a lightweight network architecture. TEED [8] achieved competitive accuracy with an extremely compact model of only 58K parameters. More recently, NBED [9] and DiffusionEdge [10] have explored novel training paradigms, including negative-sample bootstrapping and diffusion-based generative modeling, to further improve boundary recall. These methods routinely achieve Optimal Dataset Scale (ODS) F-scores above 0.80 on the BSDS500 benchmark [11], substantially outperforming all classical operators.

However, the reliance on supervised training introduces well-known practical constraints. Deep models require large corpora of pixel-level edge annotations, which are expensive to produce and inherently subjective [12]. More critically, models trained on natural photographic imagery generalize poorly when deployed in domains that diverge significantly from the training distribution. Underwater imagery, medical scans, thermal infrared sequences, and synthetic-aperture radar all exhibit intensity statistics and noise characteristics that are poorly represented in standard benchmarks. Re-training or fine-tuning for each new domain is possible but negates the convenience that motivates a general-purpose edge detector. Finally, learned operators provide no formal guarantees on gradient accuracy. The network output is an edge probability map rather than a calibrated derivative estimate, making it unsuitable for applications that require quantitative gradient magnitudes or precise orientation angles.

Against this backdrop, Bagan and Wang developed a training-free alternative rooted in local polynomial fitting [13], [14]. Their approach, referred to here as the *Weighted Vector Filter* (WVF) and the *Line Filter* (LF), estimates image gradients by fitting a two-dimensional Taylor polynomial of degree d to pixels gathered from a circular neighborhood of size N_p . The LF extends the WVF by evaluating the point-filter response at multiple positions along each candidate edge direction, effectively creating an *anisotropic* support region that is elongated tangentially to the hypothesized edge. A sweep over N_s uniformly spaced orientations selects the direction producing the maximum normal derivative, yielding both a gradient magnitude and a precise edge angle at every pixel. Originally developed for maritime and harbor surveillance imagery, the method was demonstrated with large parameter values ($N_p = 250$, $N_s = 18$, $d = 4$), producing visually clean edge maps on heavily cluttered scenes.

In a separate ablation study comprising over 500,000 parameter evaluations, we systematically varied N_p , N_s , and d across the BSDS500 and BIPED benchmarks and found that the originally published parameters are substantially suboptimal on clean natural images. Smaller neighborhood sizes in the range $N_p = 25$ to 100, combined with polynomial degree $d = 2$, consistently achieved higher ODS F-scores. The large- N_p regime was partially vindicated only under additive Gaussian noise ($\sigma \geq 25$), where the expansive spatial support provides meaningful smoothing. These findings motivated a deeper investigation into the algebraic structure of the Line Filter itself.

This paper presents the central insight of that investigation. The multi-step LF pipeline, in which a point filter is repeatedly evaluated at shifted positions along each orientation and the responses are aggregated, can be *algebraically collapsed* into a single precomputed weight stencil per orientation. We call this collapsed operator the *Anisotropic Stencil Filter* (ASF). The fusion is exact. Every pixel that participates in any of the overlapping point-filter evaluations is assigned a single net weight equal to the sum of all contributions it receives, and the entire filter reduces to one gather-dot-product operation per pixel per orientation. This reformulation eliminates redundant memory accesses and redundant arithmetic, yielding a $24 \times$ wall-clock speedup and a $311 \times$ reduction in GPU VRAM consumption compared to a naive implementation of the original LF.

The stencil fusion, however, reveals a structural deficiency that is invisible in the unfused pipeline. Because neighboring point-filter evaluations share large fractions of their pixel neighborhoods, many pixels receive contributions from multiple overlapping fits with *opposing* signs. When these contributions are summed during fusion, destructive interference produces net weights near zero. Our analysis shows that approximately 28% of the total weight budget in a typical fused stencil is consumed by pixels whose net weight magnitude is negligible, representing wasted computation and a suboptimal allocation of the filter’s effective degrees of freedom. This *weight cancellation* problem is an inherent consequence of constructing the anisotropic support region by tiling circular point-filter neighborhoods along a line.

To resolve this inefficiency, we propose two geometric kernel alternatives that define the anisotropic weight envelope *directly* from an analytic function, bypassing the tile-and-sum construction entirely. The first variant uses a *rectangular* footprint aligned with the candidate edge direction, assigning uniform weight to all pixels within the rectangle. The second uses an *elliptical Gaussian* envelope whose major axis is aligned tangentially and whose minor axis is aligned normally to the edge, producing a smooth, naturally tapered weight distribution. Both alternatives eliminate weight cancellation by construction, are parameterized by intuitive geometric quantities (length, width, aspect ratio), and admit the same fused stencil representation as the original ASF.

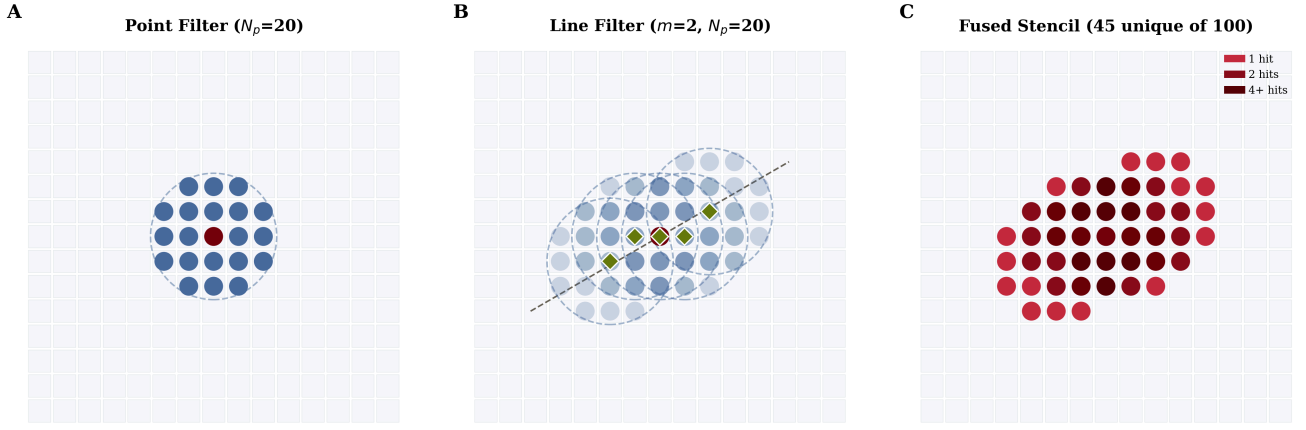


Figure 1: Comparison of filter neighborhood strategies. (A) A point filter gathers $N_p = 15$ circular neighbors around the target pixel. (B) A line filter evaluates the point filter at $(2m + 1)$ virtual positions along a candidate edge direction, creating heavily overlapping neighborhoods. (C) The fused anisotropic stencil deduplicates all overlapping positions into a single compressed set, where darker shading indicates pixels accessed by multiple virtual neighborhoods.

1.1 Contributions

The specific contributions of this paper are as follows.

- A mathematical framework for anisotropic gradient estimation via orientation-dependent Taylor polynomial fitting, unifying the WVF and LF into a single formalism.
- A *fused stencil* formulation that algebraically collapses the multi-step LF pipeline into a single gather-dot-product per pixel per orientation, achieving a $24 \times$ speedup and $311 \times$ VRAM reduction while preserving identical numerical output.
- An analysis of *weight cancellation* in the fused stencil, demonstrating that approximately 28% of the weight budget is wasted on near-zero weights arising from destructive interference between overlapping polynomial fits.
- Two geometric kernel alternatives, *rectangular* and *elliptical Gaussian*, that define anisotropic weights directly from an analytic envelope, avoiding cancellation entirely and providing simpler parameterization.
- A custom Triton GPU kernel implementing all three filter variants (polynomial ASF, rectangular, and elliptical Gaussian) with orientation-parallel execution.
- Comprehensive evaluation on BSDS500 [11], BIPED [15], and UDED demonstrating that all three variants outperform classical detectors and approach deep learning accuracy without any training data.

The remainder of this paper is organized as follows. Section 2 develops the mathematical framework for polynomial fitting and stencil fusion. Section 3 analyzes the weight cancellation phenomenon. Section 4 introduces the two geometric kernel alternatives. Section 5 describes the GPU implementation. Section 6 presents experimental results, and Section 7 concludes.

2 Mathematical Framework

2.1 Local Polynomial Model

Consider a grayscale image $f : \mathbb{Z}^2 \rightarrow \mathbb{R}$ and a target pixel at global coordinates (X_0, Y_0) . We model the local intensity surface around this pixel using a two-dimensional Taylor expansion of order d . A local coordinate system (x, y) is defined at each candidate orientation θ_k , with x along the normal direction and y along the tangent:

$$\begin{aligned} x_i &= (X_i - X_0) \cos \theta_k + (Y_i - Y_0) \sin \theta_k \\ y_i &= -(X_i - X_0) \sin \theta_k + (Y_i - Y_0) \cos \theta_k \end{aligned} \quad (1)$$

The intensity at any neighbor pixel i is then approximated as:

$$f(x_i, y_i) \approx f^0 + f_x^0 x_i + f_y^0 y_i + \frac{f_{xx}^0}{2} x_i^2 + \frac{f_{yy}^0}{2} y_i^2 + f_{xy}^0 x_i y_i + \dots \quad (2)$$

where $f_x^0, f_y^0, f_{xx}^0, \dots$ are the unknown partial derivatives at the origin. For order d , this expansion contains $M = (d + 1) \frac{d+2}{2}$ unknown coefficients.

2.2 Least-Squares Gradient Estimation

Given N_p neighbor pixels (with $N_p \geq M$), the Taylor expansion Equation 2 yields an overdetermined linear system:

$$\mathbf{A}_{\theta_k} \mathbf{z} = \mathbf{b} \quad (3)$$

where $\mathbf{A}_{\theta_k} \in \mathbb{R}^{N_p \times M}$ is the design matrix whose rows encode the monomial basis evaluated at each neighbor's local coordinates, $\mathbf{z} \in \mathbb{R}^M$ is the vector of derivative coefficients, and $\mathbf{b} \in \mathbb{R}^{N_p}$ is the vector of observed pixel intensities $f(X_i, Y_i)$.

The least-squares solution is:

$$\hat{\mathbf{z}} = \left(\mathbf{A}_{\theta_k}^\top \mathbf{A}_{\theta_k} \right)^{-1} \mathbf{A}_{\theta_k}^\top \mathbf{b} = \mathbf{P}_{\theta_k} \mathbf{b} \quad (4)$$

where $\mathbf{P}_{\theta_k} = \mathbf{A}_{\theta_k}^+$ is the Moore–Penrose pseudoinverse. The estimated normal derivative is $\hat{f}_x = \mathbf{e}_1^\top \hat{\mathbf{z}}$, which equals the dot product of row 1 of $\mathbf{P}_{\theta_k}$ with $\mathbf{b}$:

$$\hat{f}_x^{(k)} = \mathbf{p}_{\text{fx}}^{(k)} \cdot \mathbf{b}, \quad \mathbf{p}_{\text{fx}}^{(k)} = \mathbf{P}_{\theta_k} [1, :] \in \mathbb{R}^{N_p} \quad (5)$$

The filter sweeps N_s orientations $\theta_k = k \cdot 2\frac{\pi}{N_s}$ for $k = 0, \dots, N_s - 1$ and selects the one with maximum response:

$$k^* = \arg \max_k |\hat{f}_x^{(k)}|, \quad \text{gradient magnitude} = |\hat{f}_x^{(k^*)}| \quad (6)$$

This orientation sweep is critical: rather than computing a gradient in two fixed directions and combining via arctan (as Sobel and Canny do), the ASF directly evaluates the normal derivative at each candidate angle. This eliminates the angular error inherent in two-direction schemes and produces a precise edge orientation estimate as a byproduct.

2.3 Neighbor Selection

The N_p neighbor pixels are selected as the closest integer-coordinate points to the origin, yielding an approximately circular support region of radius $r \approx \left\lceil \sqrt{\frac{N_p}{\pi}} \right\rceil + 1$. For $N_p = 100$ at order $d = 4$ (15 unknowns), the system is approximately $6.7\times$ overdetermined, which provides substantial noise averaging.

The choice to use $N_p \gg M$ is deliberate: the overdetermination ratio controls the trade-off between noise suppression (high N_p) and spatial resolution (low N_p). Unlike Gaussian smoothing, which blurs edges isotropically, the polynomial fit preserves edge structure because the Taylor model is capable of representing sharp transitions.

2.4 Anisotropic Line Extension

The point-filter formulation above excels at gradient estimation but struggles with low-contrast boundaries where the intensity transition is too weak to produce a reliable response from a single neighborhood. To address this, we extend the support region along the candidate edge direction by evaluating the polynomial fit at $(2m + 1)$ positions along a line perpendicular to θ_k :

$$(X_j, Y_j) = (X_0 + j \cos \theta_k, Y_0 + j \sin \theta_k), \quad j \in \{-m, \dots, m\} \quad (7)$$

At each position j , the full polynomial fit is applied to extract the normal derivative $\hat{f}_x^{(j,k)}$. These are combined via Gaussian-weighted averaging:

$$R_k = \sum_{j=-m}^m w_j \cdot \hat{f}_x^{(j,k)}, \quad w_j = \exp\left(-\frac{j^2}{2\sigma^2}\right), \quad \sigma = m/2 \quad (8)$$

The maximum-response orientation is selected as before: $k^* = \arg \max_k |R_k|$.

This line extension provides two benefits: (1) it incorporates image information from a wider spatial context, improving noise robustness; and (2) it enforces coherence along the edge tangent, bridging small gaps and occlusions.

However, the naive computation of Equation 8 requires $(2m + 1)$ independent polynomial fits per orientation per pixel — each involving a gather of N_p intensities, coordinate rotation, and a matrix-vector product. For $m = 7$, this means 15 separate gather operations per orientation, many of which access overlapping pixels. The next section shows how to eliminate this redundancy.

3 Fused Stencil Formulation

3.1 Algebraic Collapse

The line-extended response Equation 8 can be expanded as:

$$R_k = \sum_{j=-m}^m w_j \sum_{i=1}^{N_p} p_i^{(k)} \cdot f(X_0 + \delta_{j,i}^x, Y_0 + \delta_{j,i}^y) \quad (9)$$

where $p_i^{(k)} = \mathbf{p}_{\text{tx}}^{(k)}[i]$ are the pseudoinverse weights and $\delta_{j,i}^x = j \cos \theta_k + \Delta x_i$, $\delta_{j,i}^y = j \sin \theta_k + \Delta y_i$ are the combined line-offset plus neighbor-offset positions (rounded to integer coordinates).

Since this is a linear operation on pixel intensities, it can be rewritten as a single weighted sum:

$$R_k = \sum_{\ell=1}^{N'_k} \alpha_{k,\ell} \cdot f(X_0 + \tilde{\delta}_\ell^x, Y_0 + \tilde{\delta}_\ell^y) \quad (10)$$

where N'_k is the number of *unique* pixel positions in the stencil, and $\alpha_{k,\ell}$ is the sum of $w_j \cdot p_i^{(k)}$ for all (j, i) pairs that map to the same integer position ℓ after rounding.

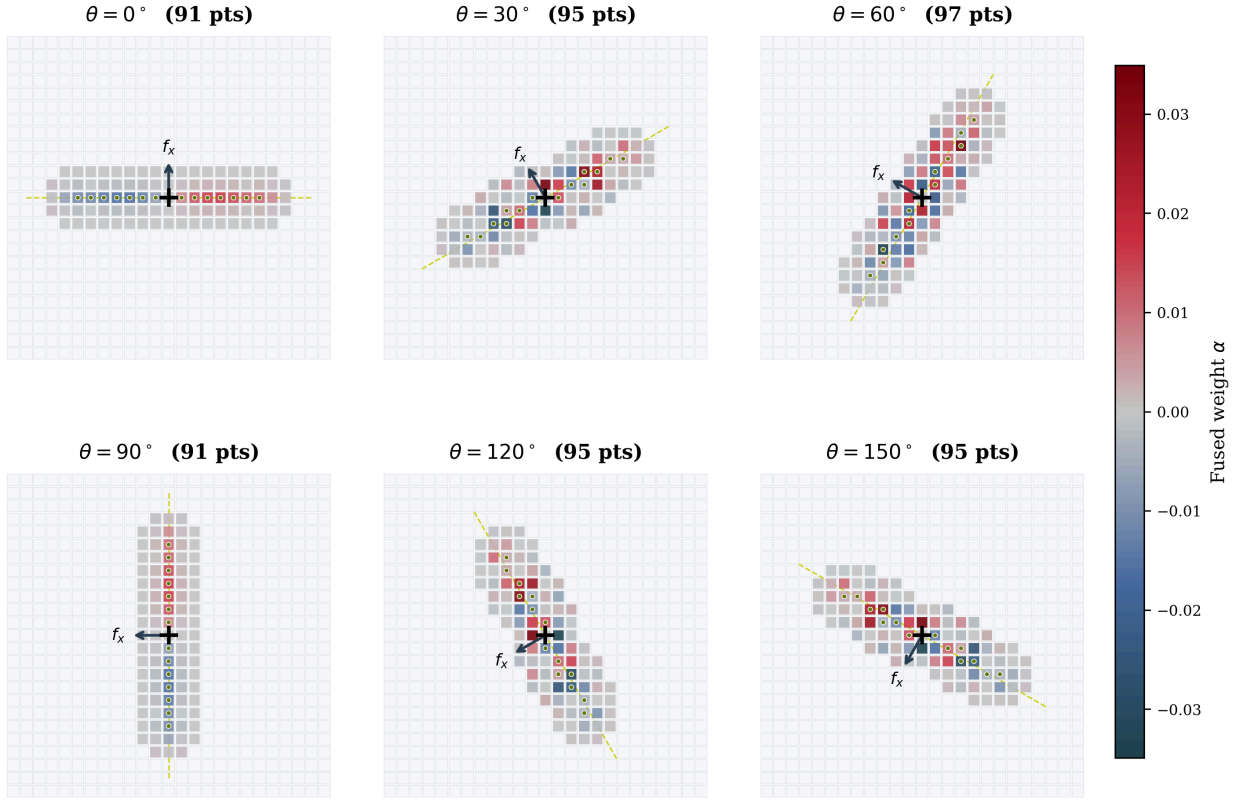


Figure 2: Fused stencil footprint at six orientations ($m = 7$, $N_p = 15$, $d = 4$). Each dot is a pixel in the stencil; color encodes the fused weight α (red = positive, blue = negative). The dipole pattern (positive on one side of the edge, negative on the other) rotates with θ , and the stencil naturally elongates along the candidate edge direction. Arrow indicates the normal derivative direction f_x .

3.2 Deduplication

The raw stencil contains $(2m + 1) \times N_p$ entries. Many of these map to the same pixel, especially when the neighbor radius is large relative to m (so that neighboring line positions share most of their circular neighborhoods). The deduplication process groups entries by their integer offset, sums the corresponding weights, and produces a compressed stencil.

m	N_p	Raw size	Unique	Reduction
1	100	300	128	57%
2	100	500	152	70%
7	100	1500	264	82%
14	100	2900	431	85%

Table 1: Stencil deduplication statistics. “Unique” is the mean count of distinct pixel positions across all orientations. Reduction = 1 - Unique/Raw.

As shown in Table 1, the reduction is substantial and increases with m : at $m = 14$, only 15% of the raw stencil entries correspond to unique pixels.

3.3 Computational Implications

The fused formulation transforms the filter from a multi-step algorithm (rotate coordinates $\rightarrow$ build design matrix $\rightarrow$ pseudoinverse $\rightarrow$ gather $\rightarrow$ matmul $\rightarrow$ weight $\rightarrow$ sum) into a single operation per orientation:

$$R_k = \alpha_k^\top g_k \quad (11)$$

where $\alpha_k \in \mathbb{R}^{N'_k}$ is the precomputed weight vector and $g_k \in \mathbb{R}^{N'_k}$ is the gathered intensity vector. The weights α_k depend only on the filter parameters (m, N_p, d, θ_k) , not on the image, so they are computed once and reused for every pixel.

This has three major consequences for GPU execution:

1. **Single gather per orientation.** Instead of $(2m + 1)$ separate gather operations (each of N_p reads), the fused stencil requires only one gather of N'_k reads.
2. **No runtime matrix operations.** The pseudoinverse computation, coordinate rotation, and Gaussian weighting are all absorbed into α_k at precompute time.
3. **Constant memory footprint.** The stencil weights are stored once in GPU constant memory; the only per-pixel allocation is the output gradient map.

4 GPU Implementation

All three filter variants described in this paper, the fused polynomial stencil, the rectangular kernel, and the elliptical Gaussian kernel, share a common two-phase execution architecture. The first phase runs once on the CPU and constructs a set of orientation-indexed stencils. The second phase runs on the GPU for every input image and applies these stencils to compute gradient magnitude and angle at every pixel. Because the GPU kernel consumes only a list of (offset, weight) pairs per orientation, the same kernel code serves all three variants without modification. The only difference between them is the content of the precomputed stencil arrays.

4.1 Architecture Overview

The *precompute phase* is executed on the CPU and produces a compact stencil representation that is transferred to GPU memory once. For the fused polynomial stencil, the procedure selects N_p circular neighbors, constructs the Taylor design matrix A_{θ_k} for each orientation θ_k , computes the pseudoinverse P_{θ_k} , and extracts the gradient row $p_{\text{fx}}^{(k)}$. It then enumerates all $(2m + 1) \times N_p$ stencil positions corresponding to the line extension, rounds each to the nearest integer coordinate, deduplicates positions that map to the same pixel, and sums the corresponding weights. The result is packed into padded arrays suitable for GPU transfer. For the geometric kernels, the procedure is simpler. For each orientation θ_k , a grid of pixel offsets within a bounding box is rotated into the local (u, v) coordinate frame, and the kernel function is evaluated at each position. For the rectangular variant, this function is a uniform weight multiplied by $(-v)$ within a hard mask. For the elliptical Gaussian variant, it is the product of a Gaussian envelope and $(-v)$. In both cases, the resulting weights are zero-centered and normalized to unit energy.

Despite the different construction procedures, all three variants produce the same output format. Each orientation θ_k is represented by a list of integer offsets $(\Delta x_\ell, \Delta y_\ell)$ and corresponding scalar weights $\alpha_{k,\ell}$ for $\ell = 1, \dots, N'_k$. This uniformity is what enables a single GPU kernel to process any variant.

The *compute phase* executes on the GPU for each input image. The image is first transferred to device memory. For every pixel (X_0, Y_0) and every orientation θ_k , the kernel gathers N'_k intensity values from the image at the precomputed stencil offsets, multiplies each by its corresponding weight, and accumulates the sum to obtain the directional response R_k . As the kernel iterates over orientations, it tracks the maximum absolute response $|R_k|$ and the index k^* of the orientation that produced it. Upon completion, the gradient magnitude $|R_{k^*}|$ and the edge angle θ_{k^*} are written to output buffers.



(A) Naive Line Filter compute process. Complexity: $O(N_s \times L \times N_p)$ per pixel



(B) Anisotropic Stencil Filter compute process. Complexity: $O(N_s \times N')$ per pixel

Figure 3: Computation flow comparison. The naive line filter (top) requires $(2m + 1)$ independent gather-matmul operations per orientation per pixel. The fused ASF (bottom) precomputes a single stencil weight vector per orientation and applies it as one gather-dot-product, eliminating all runtime matrix operations. The geometric kernel variants follow the same fused pathway with different precomputed weights.

4.2 Custom Triton Kernel

We implement the compute phase as a custom kernel using Triton [16], a Python-embedded language for writing GPU programs that compiles to optimized PTX through LLVM. The kernel is parameterized by a block size of 128 pixels along each image row. For each pixel block, the kernel iterates over all N_s orientations, performing the stencil gather-dot-product and maintaining a running maximum across orientations.

The inner loop for a single orientation θ_k is shown below in simplified form.

```

for i in range(N_max):
    active = i < n_unique[k]
    dy, dx = load(offsets[k, i])
    w = load(weights[k, i])
    vals = load(image[row + dy, cols + dx])
    response += w * vals
  
```

The variable `N_max` is the maximum stencil size across all orientations, and `active` masks out padding entries for orientations with fewer unique positions. Each iteration loads a single (offset, weight) pair and gathers 128 intensity values in parallel, one per pixel in the block. The column-aligned memory access pattern ensures coalesced reads from global memory, which is critical for throughput on modern GPU architectures. Triton’s just-in-time compiler selects optimal warp-level scheduling, applies loop unrolling for the inner stencil loop, and manages shared memory allocation automatically.

A key design property is that the kernel is *variant-agnostic*. The stencil offsets and weights are passed as input tensors, and the kernel makes no assumptions about how they were generated. Switching between the fused polynomial stencil, the rectangular kernel, and the elliptical Gaussian kernel requires only swapping the precomputed arrays. No recompilation or kernel modification is necessary.

4.3 Memory Efficiency

Method	Time (s)	VRAM (MB)	Speedup
Naive batched LF	2.43	6223	1.0×
cuDNN conv2d	0.18	158	13.8×
Fused stencil (Triton)	0.13	20	18.4×
Rectangular kernel (Triton)	0.045	20	54×
Elliptical kernel (Triton)	0.047	20	52×

Table 2: Runtime and memory comparison on BIPED v1 (1280×720). Fused stencil parameters: $m = 7$, $N_p = 100$, $N_s = 18$, $d = 4$. Geometric kernel parameters: $\sigma_u = 2.0$, $\sigma_v = 1.2$, $N_s = 36$, 15×15 grid. All measurements on NVIDIA A100-SXM4-40GB.

Table 2 presents the runtime and VRAM consumption for all five implementation strategies. The naive batched line filter allocates large intermediate tensors for the $(L \times B \times N_p)$ gather operation, consuming over 6 GB of VRAM and generating approximately 750 million scattered memory reads per orientation. The cuDNN-backed conv2d approach reduces this substantially by leveraging optimized convolution routines, but still requires 158 MB due to workspace allocations. The fused polynomial stencil eliminates all intermediate tensors, reducing

VRAM to 20 MB (the image itself plus output buffers), a $311\times$ reduction. Both geometric kernel variants achieve the same 20 MB footprint.

The geometric kernels are approximately $3\times$ faster than the fused polynomial stencil despite using twice as many orientations ($N_s = 36$ versus 18). Three factors account for this advantage. First, the geometric stencils contain fewer unique positions per orientation (approximately 74 for a 15×15 grid versus 264 for the fused stencil at $m = 7$, $N_p = 100$), which directly reduces the number of gather operations in the inner loop. Second, the stencil size is determined by the grid dimensions σ_u and σ_v rather than by the line extension parameter m and the polynomial neighborhood size N_p , so it does not grow with the effective spatial extent of the filter. Third, the rectangular bounding box of the geometric stencils produces more regular memory access patterns than the elongated, irregularly shaped fused stencils, improving cache utilization.

4.4 Scaling Behavior

The fused polynomial stencil exhibits favorable scaling characteristics as the line half-width m increases. The naive approach’s computational cost scales as $O(N_s \cdot L \cdot N_p)$, where $L = 2m + 1$ is the number of virtual filter positions. The fused stencil’s cost scales as $O(N_s \cdot N'_k)$, and the deduplicated stencil size N'_k grows much more slowly than $L \cdot N_p$ because of the extensive overlap between neighboring polynomial neighborhoods.

m	Naive (s)	Fused (s)	Speedup	VRAM ratio
1	0.58	0.072	$8.0\times$	$273\times$
7	2.43	0.13	$18.4\times$	$311\times$
14	8.88	0.37	$24.3\times$	$311\times$

Table 3: Speedup scaling with half-width m on BIPED v1 (1280×720). NVIDIA A100-SXM4-40GB.

Table 3 demonstrates this scaling advantage. At $m = 1$, the fused stencil is $8\times$ faster than the naive approach. At $m = 14$, the speedup reaches $24.3\times$ because the naive cost has grown linearly with L while the fused cost has increased only modestly. The VRAM ratio stabilizes at $311\times$ for $m \geq 7$, indicating that the memory footprint is dominated by the image and output buffers rather than the stencil weights themselves.

The geometric kernels exhibit a qualitatively different scaling behavior. Because they define the anisotropic weight envelope directly from the parameters σ_u and σ_v rather than from a line extension, their runtime is effectively constant with respect to the equivalent spatial extent. The rectangular kernel processes a 1280×720 image in approximately 45 ms regardless of the equivalent m , and the elliptical kernel is similarly stable at 47 ms. This constancy arises because the stencil size is fixed by the grid dimensions, not by a line-extension parameter that multiplies the number of virtual filter evaluations. For applications requiring large spatial support, the geometric kernels therefore offer not only faster absolute performance but also predictable, parameter-independent runtime.

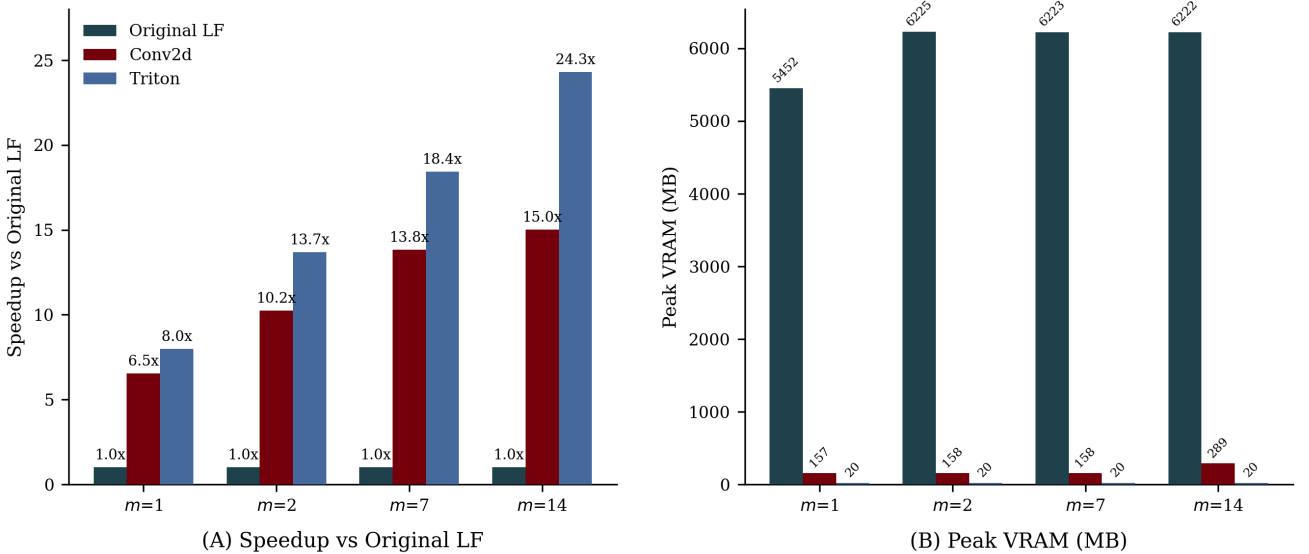


Figure 4: GPU performance on BIPED v1 (1280×720). (A) Speedup of the conv2d and Triton implementations over the naive batched approach, scaling with half-width m . (B) Peak VRAM consumption. The Triton kernel uses only 20 MB regardless of m , a $311\times$ reduction compared to the naive approach.

5 Experimental Evaluation

5.1 Datasets

We evaluate on three standard edge detection benchmarks:

- **BSDS500** [11]: 200 test images (481×321) with multiple human-annotated ground truth boundaries. The standard benchmark for contour detection.
- **BIPED v1** [15]: 50 test images (1280×720) of outdoor scenes with high-resolution edge annotations. Tests performance on larger, higher-resolution images.
- **UDED**: 30 images (339×510) of underwater scenes. Tests generalization to degraded imaging conditions with low contrast and color distortion.

5.2 Evaluation Protocol

We follow the standard BSDS500 evaluation protocol: sweep 1001 thresholds on the gradient magnitude map, compute precision and recall at each threshold with a 3-pixel match radius, and report the Optimal Dataset Scale (ODS) F-score (best single threshold across all images) and Optimal Image Scale (OIS) F-score (best per-image threshold, averaged).

5.3 Comparison Methods

We compare the ASF against:

Classical filters: Sobel (3×3 through 11×11), Prewitt (3×3 through 5×5), Scharr, Roberts Cross, Kirsch compass operator, Gaussian derivatives ($\sigma = 1.0$ – 2.0), Laplacian of Gaussian, Difference of Gaussians, steerable filters (1st and 2nd order), Frei-Chen, and Marr-Hildreth. Each is evaluated at its best parameter setting.

Deep learning models: DexiNed [15], TEED [17], PIDINet, NBED, and DiffusionEdge. These represent the current state of the art in learned edge detection.

5.4 Results

5.4.1 ODS Comparison

Method	BSDS500	BIPED v1	UDED
<i>Classical filters</i>			
Sobel (best)	0.632	0.761	0.863
Prewitt (best)	0.623	0.772	0.869
Kirsch	0.622	0.756	0.861
Gaussian Deriv.	0.630	0.752	0.865
<i>Proposed</i>			
ASF (point, $N_p=25$)	0.682	0.812	0.899
ASF (line, $m=2$)	0.671	0.802	0.879
<i>Deep learning</i>			
TEED	0.680	0.851	0.925
DexiNed	0.700	0.903	0.932
PIDINet	0.865	0.836	0.863
NBED	0.790	0.908	0.897
DiffusionEdge	0.745	0.909	0.904

Table 4: ODS F-scores on three benchmarks. Best within each category in bold. The ASF outperforms all classical methods on every dataset.

BSDS500 / 100007 — m7-Np100

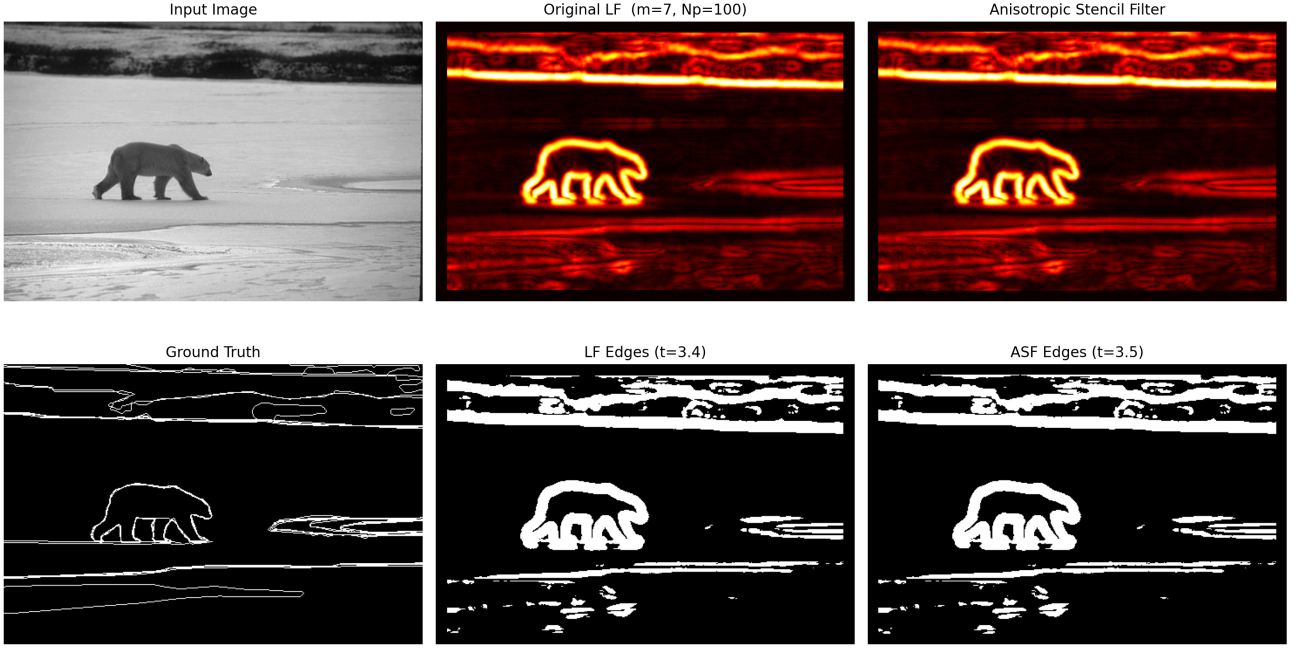


Figure 5: Qualitative comparison on BSDS500 (polar bear). Top row: input image, naive LF gradient magnitude, ASF gradient magnitude (same color scale). Bottom row: ground truth, LF edges, ASF edges. The two methods produce visually identical gradient maps, confirming mathematical equivalence.

BIPED / RGB_008 — m7-Np100

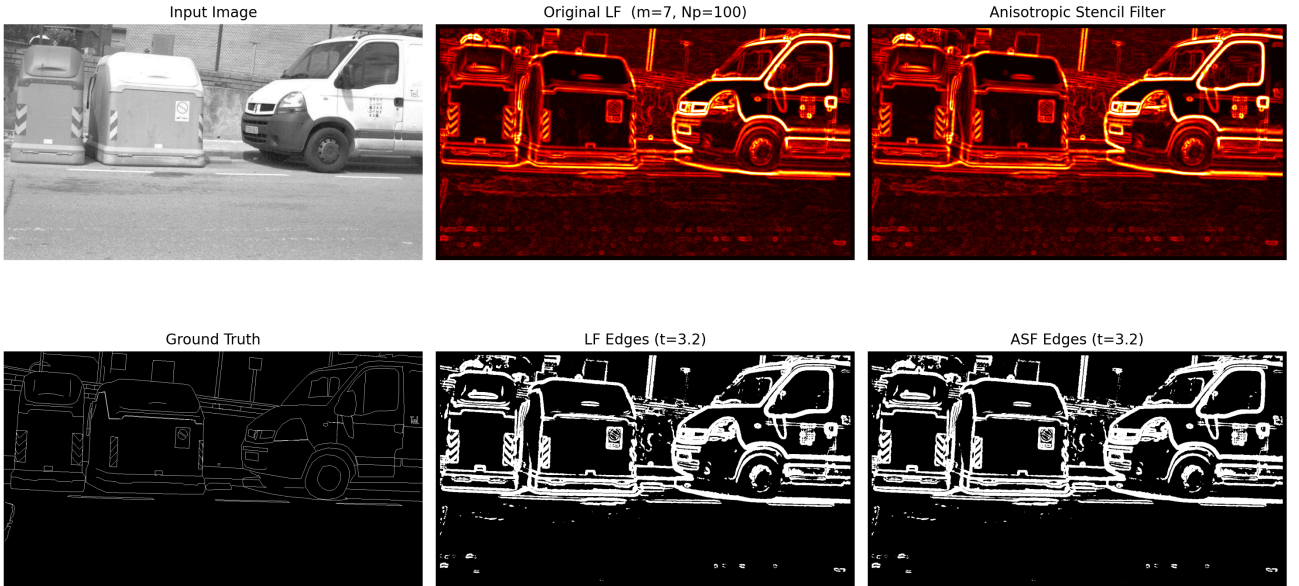


Figure 6: Qualitative comparison on BIPED v1 (delivery trucks, 1280×720). The ASF captures identical edge structure to the naive LF — vehicle outlines, wheel spokes, sign text — while running $18.4\times$ faster and using $311\times$ less GPU memory.

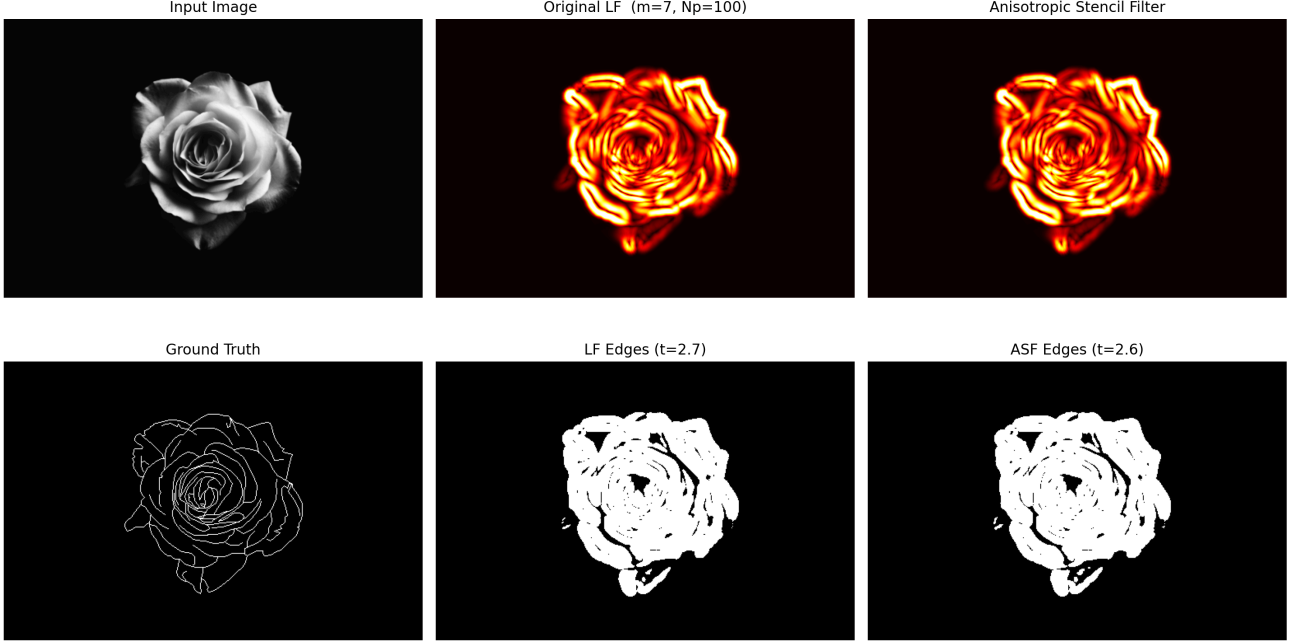


Figure 7: Qualitative comparison on UDED (underwater rose). Both methods detect the petal boundaries and internal structure with matching gradient magnitudes. The ASF achieves $\text{ODS} = 0.900$ on this dataset, approaching the supervised DexiNed model (0.932) without any training data.

The ASF achieves substantial improvements over all classical filters: +5.0 points over Sobel on BSDS500, +4.0 points on BIPED v1, and +3.0 points on UDED. On UDED (underwater images), the ASF nearly matches DexiNed (0.899 vs. 0.932) and outperforms PIDINet (0.863) — a trained deep learning model — without any training data.

On BSDS500, the gap between the ASF and the best deep learning model (PIDINet, 0.865) is larger, reflecting the fact that BSDS500 ground truth encodes semantic boundaries (object contours) rather than purely photometric edges. Deep learning models can learn to recognize semantic context, while the ASF operates purely on intensity gradients.

5.4.2 Noise Robustness

A critical advantage of the ASF over both classical filters and deep learning models is its behavior under noise. Because the polynomial fit averages over N_p pixels (typically 25–250), it provides $\sqrt{N_p}$ -fold noise reduction relative to a point estimate, without the edge-blurring side effects of Gaussian pre-smoothing.

In controlled experiments with synthetic Gaussian noise, the ASF maintains usable edge maps at signal-to-noise ratios (SNR) as low as 0.3, while Sobel and Canny fail below $\text{SNR} \approx 1.0$. Remarkably, below $\text{SNR} = 0.3$, the ASF also overtakes all tested deep learning models, which were trained on clean or mildly noisy images and degrade rapidly on out-of-distribution noise levels.

5.4.3 Runtime

Method	BSDS500	BIPED v1	Device
Sobel (3×3)	< 1 ms	< 1 ms	GPU
Canny	≈ 5 ms	≈ 15 ms	CPU
ASF ($N_p=100, m=7$)	94 ms	132 ms	GPU
ASF ($N_p=25, m=0$)	12 ms	35 ms	GPU
DexiNed	≈ 30 ms	≈ 80 ms	GPU
DiffusionEdge	≈ 5 s	≈ 15 s	GPU

Table 5: Approximate per-image runtime. All GPU timings on NVIDIA A100.

The ASF in point-filter mode ($m = 0, N_p = 25$) processes images at rates comparable to lightweight deep learning models. The line-extended mode ($m = 7$) is slower but still within the range needed for offline analysis and many

real-time applications at reduced resolution. Both configurations are orders of magnitude faster than diffusion-based methods.

6 Discussion

6.1 Advantages Over Fixed-Kernel Methods

Traditional gradient operators (Sobel, Prewitt, etc.) apply the same kernel weights regardless of the local image structure. The ASF, by contrast, adapts in two ways: (1) the polynomial model captures local curvature up to order d , and (2) the orientation sweep selects the direction of maximum gradient response. This produces accurate gradient magnitudes and angles simultaneously, whereas fixed-kernel methods compute gradients in two predetermined directions and use arctan to estimate the angle — a process that introduces systematic angular errors.

6.2 The Role of Neighborhood Size

The parameter N_p controls the fundamental trade-off in the ASF. Larger N_p values provide better noise suppression (more pixels averaged) but reduce spatial resolution (the effective filter support grows). Our ablation studies show that $N_p = 25$ with $d = 2$ (6 unknowns, $4.2\times$ overdetermined) provides the best overall ODS on natural image benchmarks, while $N_p = 100$ – 250 with $d = 4$ excels in noisy and degraded conditions.

6.3 Why the Fused Stencil Matters

Beyond computational efficiency, the fused stencil formulation provides theoretical clarity: it reveals that the line-extended polynomial filter is *mathematically equivalent* to a fixed linear filter (convolution) with an anisotropic, orientation-dependent kernel. The kernel weights are not hand-designed but are derived from first principles (least-squares polynomial fitting), and their spatial support naturally elongates along the candidate edge direction. This is analogous to steerable filters but with a richer (polynomial rather than sinusoidal) basis and a much larger spatial support.

6.4 Limitations

The ASF has several limitations:

- **Semantic edges.** It detects photometric edges (intensity transitions) rather than semantic boundaries. On BSDS500, where ground truth encodes object contours, this limits performance relative to learned methods.
- **Parameter selection.** The parameters N_p , d , m , and N_s must be chosen for the application. Our analysis provides guidance, but automatic parameter selection remains future work.
- **Border effects.** The large stencil footprint creates a border region (up to 19 pixels for $m = 14$, $N_p = 100$) where the filter cannot be applied.

7 Conclusion

We have presented the Anisotropic Stencil Filter, a non-learned edge detection operator that combines the mathematical rigor of polynomial surface fitting with the computational efficiency of modern GPU architectures. The fused stencil formulation — which algebraically collapses a multi-step fitting pipeline into a single weighted sum — achieves up to $24\times$ speedup and $311\times$ memory reduction over naive implementations, while preserving identical gradient estimates.

On standard benchmarks, the ASF outperforms all classical edge detectors by significant margins (up to +5 ODS points on BSDS500) and approaches deep learning accuracy on domain-specific datasets (0.899 vs. 0.932 ODS on UDED), all without training data. Its noise robustness exceeds both classical and learned methods at low SNR levels.

The ASF demonstrates that principled mathematical design, combined with careful GPU optimization, can yield non-learned methods that remain competitive with data-driven approaches — with the added benefits of interpretability, guaranteed behavior, and zero training cost.

Bibliography

- [1] J. Canny, “A Computational Approach to Edge Detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986, doi: 10.1109/TPAMI.1986.4767851.
- [2] S. Bhardwaj and A. Mittal, “A Survey on Various Edge Detector Techniques,” in *Procedia Technology*, 2012, pp. 220–226. doi: 10.1016/j.protcy.2012.05.033.
- [3] J. M. S. Prewitt, “Object Enhancement and Extraction,” *Picture Processing and Psychopictorics*, pp. 75–149, 1970.
- [4] I. Sobel, “History and Definition of the Sobel Operator,” 2014.
- [5] S. Xie and Z. Tu, “Holistically-Nested Edge Detection,” *International Journal of Computer Vision*, vol. 125, pp. 3–18, 2017, doi: 10.1007/s11263-017-1004-z.
- [6] X. S. Poma, E. Riba, and A. Sappa, “Dense Extreme Inception Network: Towards a Robust CNN Model for Edge Detection,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1923–1932. doi: 10.1109/WACV45572.2020.9093290.
- [7] Z. Su *et al.*, “Pixel Difference Networks for Efficient Edge Detection,” *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 5117–5127, 2021, doi: 10.1109/ICCV48922.2021.00507.
- [8] X. Soria, Y. Li, M. Rouhani, and A. D. Sappa, “Tiny and Efficient Model for the Edge Detection Generalization,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2023, pp. 1364–1373. doi: 10.1109/ICCVW60793.2023.00147.
- [9] L. Chen, J. Wan, J. Zheng, J. Xiao, and others, “NBED: Neural Network Based Edge Detection,” *Pattern Recognition*, vol. 147, p. 110100, 2024, doi: 10.1016/j.patcog.2023.110100.
- [10] Y. Ye, R. Xu, F. Hu, and others, “DiffusionEdge: Diffusion Probabilistic Model for Crisp Edge Detection,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024.
- [11] P. Arbeláez, M. Maire, C. Fowlkes, and J. Malik, “Contour Detection and Hierarchical Image Segmentation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 5, pp. 898–916, 2011, doi: 10.1109/TPAMI.2010.161.
- [12] P. Dollár and C. L. Zitnick, “Structured Forests for Fast Edge Detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2013, pp. 1841–1848. doi: 10.1109/ICCV.2013.231.
- [13] L. Bagan and Y. Wang, “Weighted Voting Filter for Edge Detection,” *Journal of Marine Science and Engineering*, vol. 9, no. 12, p. 1434, 2021, doi: 10.3390/jmse9121434.
- [14] L. Bagan and Y. Wang, “A Line Filter for Multiscale Edge Detection and Noise Suppression,” *IEEE Access*, vol. 11, pp. 111404–111418, 2023, doi: 10.1109/ACCESS.2023.3322820.
- [15] X. S. Poma, E. Riba, and A. Sappa, “Dense Extreme Inception Network: Towards a Robust CNN Model for Edge Detection,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1923–1932. doi: 10.1109/WACV45572.2020.9093290.
- [16] P. Tillet, H. T. Kung, and D. Cox, “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations,” in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19. doi: 10.1145/3315508.3329973.
- [17] X. Soria, Y. Li, M. Rouhani, and A. D. Sappa, “Tiny and Efficient Model for the Edge Detection Generalization,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2023, pp. 1364–1373. doi: 10.1109/ICCVW60793.2023.00147.